```
            default:
                throw new Exception('unsupported vehicle type');
        }
    }
}
```

Clearly the problem domain contains vehicles that are cars and buses, but the solution space currently only contains a `Vehicle` class, and the differences in behavior between cars and buses are determined in a `switch/case` statement. If the problem domain grows and we need to support, say, motorbikes, then we'll have to modify the code of `Vehicle`.

The `Vehicle` is not closed to modification. On the contrary, with this design our solution space *requires* `Vehicle` to be modified every time we introduce new vehicle types. Additionally, `Vehicle` is not open for extension; there's no way for a child class to impact the number of passengers it needs to represent. This code is difficult to maintain and not particularly robust (note the use of `Exception` in the `default` case). However, we can fix this by applying OCP to the design.

```
class Vehicle
{
    protected int $maxPassengers = 0;

    public function getMaxPassengers(): int
    {
        return $this->maxPassengers;
    }
}

class Car extends Vehicle
{
    protected int $maxPassengers = 4;
}

class Motorbike extends Vehicle
{
    protected int $maxPassengers = 2;
}
```